

Standard Operating Procedure (SOP)

Hardware Replacement and System Startup

Document Version: 1.0 **Last Updated:** 2025-11-24 **System:** IROPO (Intelligent Robot Positioning System)

Table of Contents

1. [Pre-Replacement Checklist](#)
 2. [Hardware Replacement Procedures](#)
 3. [System Configuration](#)
 4. [Calibration Procedures](#)
 5. [System Startup](#)
 6. [Verification and Testing](#)
 7. [Troubleshooting](#)
-

Pre-Replacement Checklist

Safety First

- ☐ **EMERGENCY STOP** accessible and tested
- ☐ Disconnect power to robot arm
- ☐ Ensure workspace is clear
- ☐ Backup current configuration files:

```
bash cp config.json config.json.backup cp homography_auto.pkl homography_auto.pkl.backup cp homography_det_to_robot.pkl homography_det_to_robot.pkl.backup
```

Document Current Setup

- ☐ Take photos of current hardware setup
- ☐ Note all cable connections

- [] Record current IP addresses
 - [] Note camera serial numbers (if FLIR cameras)
 - [] Save current robot position: `bash # Run this in robot controller # Record displayed position`
-

Hardware Replacement Procedures

Option A: Replacing Robot Arm

Step 1: Physical Disconnection

1. **Power down the robot**
2. Turn off robot power switch
3. Wait 30 seconds for complete shutdown
4. **Disconnect cables**
5. Ethernet cable from robot base
6. Power cable
7. Label all cables for reconnection
8. **Remove robot from mounting**
9. Unbolt robot from work surface
10. Store in safe location

Step 2: Install New Robot

1. **Mount new robot**
2. Bolt securely to work surface
3. Ensure stable and level mounting
4. **Connect cables**
5. Connect power cable
6. Connect Ethernet cable to same network switch
7. **Configure network**
8. Power on robot
9. Use xArm Studio or button on robot to set IP address

10. **Default IP to use:** 192.168.1.151
11. If different IP needed, note for configuration step
12. **Initialize robot**
13. Use xArm Studio to:
 - Clear any warnings
 - Enable servo motors
 - Test basic movement
 - Set motion mode to position mode

Step 3: Update Configuration

Go to [System Configuration - Robot Settings](#)

Option B: Replacing Cameras

Step 1: Identify Camera Type

- **Current System:** FLIR cameras (PySpin SDK)
- **If replacing with FLIR:** Skip to Step 2
- **If replacing with different brand:** See [Changing Camera Type](#)

Step 2: Physical Replacement (FLIR Cameras)

Detection Camera (Top-down view): 1. Power off camera or disconnect USB 2. Unmount camera from overhead position 3. Note mounting height and angle 4. Install new camera in **exact same position** 5. Connect USB cable 6. Connect power (if using external power)

Inspection Camera (Gripper-mounted): 1. Power off camera 2. Remove from gripper mount 3. Install new camera in **exact same position and orientation** 4. Secure all cables to prevent interference with robot motion 5. Connect USB cable

Step 3: Verify Camera Detection

```
# Test camera detection
python3 -c "import PySpin; system = PySpin.System.GetInstance(); cam_list = system.GetCameras();"
```

Expected output: Found 2 cameras

Step 4: Identify Camera Indices

```
# Run camera test
python3 << 'EOF'
import PySpin
import cv2

system = PySpin.System.GetInstance()
cam_list = system.GetCameras()

for i, cam in enumerate(cam_list):
    print(f"\nCamera Index {i}:")
    cam.Init()
    nodemap = cam.GetNodeMap()
    node_device_serial = PySpin.CStringPtr(nodemap.GetNode('DeviceSerialNumber'))
    print(f"  Serial: {node_device_serial.GetValue()}")

    # Test capture
    cam.BeginAcquisition()
    image_result = cam.GetNextImage(1000)
    print(f"  Image size: {image_result.GetWidth()}x{image_result.GetHeight()}")
    image_result.Release()
    cam.EndAcquisition()
    cam.DeInit()

cam_list.Clear()
system.ReleaseInstance()
EOF
```

Note which camera is which: - Camera 0 should be **detection camera** (top-down) - Camera 1 should be **inspection camera** (gripper-mounted)

If reversed, update `config.json` camera indices.

Step 5: Update Configuration

Go to [System Configuration - Camera Settings](#)

Option C: Changing Camera Type

WARNING: Changing from FLIR to another camera type requires code modifications.

Supported Alternatives:

- USB webcams (OpenCV compatible)
- Other industrial cameras with Python SDK

Code Modifications Required:

1. **Modify:** `/home/user/arm-app/yo1o-mouse-v2.py`
2. Replace PySpin initialization (lines ~689-730)
3. Replace camera capture method (lines in main loop)
4. **Example for USB webcam:** `python # Replace PySpin initialization with: detection_cam = cv2.VideoCapture(0) # Index 0 for detection inspection_cam = cv2.VideoCapture(1) # Index 1 for inspection`

```
# Set resolution detection_cam.set(cv2.CAP_PROP_FRAME_WIDTH, 1920)
detection_cam.set(cv2.CAP_PROP_FRAME_HEIGHT, 1080) ``
```

1. **Update requirements:**
2. Remove PySpin dependency
3. Ensure opencv-python installed

System Configuration

Configuration File Location

File: `/home/user/arm-app/config.json`

Updating Robot Settings

1. **Open configuration file:** `bash cd /home/user/arm-app nano config.json`
2. **Update robot IP address:** `json { "robot": { "ip": "192.168.1.151", // Change if different ... } }`
3. **Verify network connectivity:** `bash ping 192.168.1.151` Should see replies. Press Ctrl+C to stop.
4. **Test robot connection:** `python3 << 'EOF' from xarm.wrapper import XArmAPI`

```
arm = XArmAPI('192.168.1.151') arm.connect() print(f"Connected: {arm.connected}") print(f"State: {arm.state}") arm.disconnect() EOF ``
```

Updating Camera Settings

1. **Open configuration file:** `bash nano config.json`
2. **Update camera indices if needed:** `json { "camera": { "detection_index": 0, // Top-down
camera "inspection_index": 1, // Gripper camera ... } }`
3. **Adjust camera window positions (optional):** `json "detection_window": { "width": 960,
"height": 720, "x_position": 950, "y_position": 50 }`

Updating Workspace Settings

Only change if workspace physical dimensions changed!

```
{
  "workspace": {
    "x_min": 0,
    "x_max": 300,      // Detection workspace in mm
    "y_min": 0,
    "y_max": 300,
    "safe_height": 150,    // Safe Z height
    "pick_height": -2,     // Z when picking
    "inspection_height": 103.4 // Inspection camera height
  }
}
```

Calibration Procedures

When Calibration is Required

MUST recalibrate if: - Detection camera moved or replaced - Robot arm replaced or repositioned -
Workspace layout changed - Mounting height changed

NOT required if: - Only inspection camera replaced (same position) - Software reinstalled - Config values
changed (not hardware)

Understanding Calibration Files

The system uses **two calibration files** to transform coordinates from camera pixels to robot positions:

1. homography_auto.pkl

Purpose: Transforms camera pixel coordinates to workspace coordinates (0-300mm space)

What it does: - Converts what the camera sees (pixels) into real-world measurements (millimeters) - Maps the 300mm × 300mm detection area - Created by detecting 4 white circles at workspace corners

How it's created: - **Automatic method:** Place 4 white circles, system detects them and calculates transformation - **Manual method:** Enter pixel coordinates and corresponding mm coordinates for 4 corners - **File location:** `/home/user/arm-app/homography_auto.pkl`

Example transformation:

```
Camera sees object at pixel (1024, 768)
↓ homography_auto.pkl transforms
Object is at workspace position (150.0, 150.0) mm
```

2. homography_det_to_robot.pkl

Purpose: Transforms workspace coordinates to actual robot arm coordinates

What it does: - Converts workspace coordinates (0-300mm) to robot coordinate system - Maps workspace corners to robot reach positions - Accounts for robot base position and orientation

How it's created: - Enter workspace corners (0,0 / 300,0 / 300,300 / 0,300) - Enter corresponding robot coordinates where arm should reach - System calculates transformation matrix

Default mapping:

```
Workspace Corner → Robot Position
[0, 0]           → [88.9, 312.0]   (Bottom Left)
[300, 0]         → [88.9, 14.7]    (Bottom Right)
[300, 300]       → [382.0, 14.7]   (Top Right)
[0, 300]         → [382.0, 312.0]  (Top Left)
```

File location: `/home/user/arm-app/homography_det_to_robot.pkl`

Example transformation:

```
Object at workspace (150.0, 150.0) mm
↓ homography_det_to_robot.pkl transforms
Robot should move to (235.45, 163.35) mm
```

Complete Coordinate Flow

```

Camera Pixel (1024, 768)
  ↓ [homography_auto.pkl]
Workspace Coordinate (150.0, 150.0) mm
  ↓ [homography_det_to_robot.pkl]
Robot Coordinate (235.45, 163.35) mm
  ↓ [Robot moves to position]
Gripper reaches object

```

How to use these files: 1. Create once during initial setup or after hardware changes 2. Files are automatically loaded by both GUI and command-line modes 3. System uses them for all coordinate transformations 4. Delete and recreate if calibration becomes inaccurate

Preparation for Calibration

Materials Needed:

- 4 white circular markers (20-30mm diameter)
- Can use: white stickers, bottle caps, printed circles
- Dark/contrasting background surface
- Good lighting (no harsh shadows)

Step 1: Delete Old Calibration

```

cd /home/user/arm-app
rm homography_auto.pkl
rm homography_det_to_robot.pkl

```

Step 2: Place Calibration Markers

CRITICAL: Marker placement defines your workspace!

1. Measure 300mm x 300mm workspace

2. Place markers at corners: Marker 4 Marker 3 (0, 300) (300, 300)

```

+-----+ | | Detection Area | | 300x300mm | | |
+-----+ Marker 1 Marker 2 (0, 0) (300, 0)

```

3. Ensure markers are:

4. Clearly visible to camera
5. High contrast with background

- 6. Flat and circular
- 7. Not overlapping or touching workspace edges

Step 3: Verify Robot Corner Mapping

The robot will map these corners to physical coordinates:

Detection Corner	→	Robot Coordinate	
[0, 0]	→	[88.9, 312.0]	(Bottom Left)
[300, 0]	→	[88.9, 14.7]	(Bottom Right)
[300, 300]	→	[382.0, 14.7]	(Top Right)
[0, 300]	→	[382.0, 312.0]	(Top Left)

Ensure robot can physically reach these positions.

Running Auto-Calibration

Calibration runs automatically on first startup after deletion of calibration files.

The system will: 1. Detect the 4 white circles 2. Create coordinate transformation matrices 3. Save `homography_auto.pkl` (camera → workspace) 4. Save `homography_det_to_robot.pkl` (workspace → robot)

Success indicators: - Console shows: "Calibration completed" - Both .pkl files created - Robot moves to home position

If calibration fails: - See [Calibration Troubleshooting](#)

System Startup

Startup Options

The system can be started in **two different modes**:

Option A: GUI Application (Recommended for Setup/Calibration) - User-friendly graphical interface - Built-in calibration wizard - Visual feedback and monitoring - Easy calibration file creation - Best for: Initial setup, calibration, testing, monitoring

Option B: Command-Line Mode (Recommended for Production) - Full robot control capability - Pick and place operations - Inspection mode - Best for: Active production, automated operations

Choose **Option A** if you need to calibrate the system or just monitor detections. Choose **Option B** if you need full robot control for pick-and-place operations.

Prerequisites Check

Before starting either mode, verify: - ☐ Robot powered on and connected to network - ☐ Both cameras connected via USB - ☐ Configuration file updated (config.json) - ☐ Calibration markers in place (if new calibration needed) - ☐ Workspace clear of obstacles - ☐ Emergency stop accessible

Option A: GUI Application Startup

Use this method for calibration, setup, and monitoring.

Step 1: Start GUI Application

```
cd /home/user/arm-app
python3 arm-app-v1.1.py
```

Step 2: Configure Calibration (First Time Setup)

Tab 1: Calibration Setup

Part 1: Create homography_auto.pkl (Camera → Workspace)

This file converts camera pixels to workspace millimeters.

1. **Ensure calibration markers are in place:**
 2. Place 4 white circles (20-30mm diameter) at workspace corners
 3. Pattern: Bottom-Left, Bottom-Right, Top-Right, Top-Left
4. **Run automatic calibration:**
 5. Select "Auto Calibration (Circle Detection)" from dropdown
 6. Click "Calibrate" button
 7. System detects circles and calculates transformation
 8. Wait for confirmation: "Auto calibration completed!"
 9. **File created:** `/home/user/arm-app/homography_auto.pkl` ✓

Part 2: Create homography_det_to_robot.pkl (Workspace → Robot)

This file converts workspace coordinates to robot arm positions.

1. **Configure transformation:**
 2. Leave "Auto Calculation" checkbox enabled

3. Enter only Bottom Left (BL) and Top Right (TR) robot coordinates:

- **BL:** Workspace (0, 0) → Robot (88.9, 312.0)
- **TR:** Workspace (300, 300) → Robot (382.0, 14.7)

4. **Calculate transformation:**

5. Click "Calculate" button

6. System auto-calculates BR and TL coordinates

7. Wait for confirmation: "Robot transformation calculated!"

8. **File created:** `/home/user/arm-app/homography_det_to_robot.pkl` ✓

Both calibration files are now ready to use!

Step 3: Start Live Detection

Tab 2: Live Camera View

1. Click "**START System**" button at bottom
2. Detection camera feed appears
3. YOLO detection starts automatically
4. Objects inside workspace show in **green**
5. Objects outside workspace show in **red**
6. Detection status shown: "Not Ready" / "Detect" / "Ready"

Step 4: Monitor and Test

Features Available:

- **Single Click on Object:**
 - Shows object information panel
 - Displays: ID, position, angle, dimensions
 - Shows workspace coordinates
- **Double Click on Object:**
 - Opens inspection visualization box (36mm × 36mm)
 - Use arrow keys to rotate: ← (CCW) / → (CW)
 - Press ESC to close inspection box
- **Single Click on Empty Space:**
 - Shows clicked pixel coordinates
 - Shows workspace mm coordinates
 - Verifies calibration accuracy

Tab 3: System Info

- Shows system status and readiness percentage
- Camera connection status
- Calibration status
- Theme selection (Dark/Light mode)

Step 5: Stop System

Click "**STOP System**" button when finished.

Important Notes: - GUI is ideal for creating and verifying calibration files - Once calibration files are created, they can be used by command-line mode - Use this mode for calibration, testing, and verification

Option B: Command-Line Startup (Full Robot Control)

Use this method for production operations with robot control.

Step-by-Step Startup Sequence

Step 1: Open Two Terminal Windows

Terminal 1: Robot Controller **Terminal 2:** Vision System

Step 2: Start Robot Controller (Terminal 1)

```
cd /home/user/arm-app
python3 xarm-motion-v1.7.py
```

Expected sequence:

```
Loading configuration...
Connecting to robot at 192.168.1.151...
✓ Connected
✓ Motion enabled
✓ Errors cleared
✓ Mode set to position control
Loading homography matrices...
Moving to calibration position...
Waiting for YOLO calibration...
```

Robot will: 1. Connect and initialize 2. Move to calibration position: [-55.3, 207.3, 92.7, 179.7, -0.2, -2.4]
3. Wait for vision system calibration

Leave this terminal running!

Step 3: Start Vision System (Terminal 2)

Wait for robot to reach calibration position first!

```
cd /home/user/arm-app  
python3 yolo-mouse-v2.py
```

Expected sequence:

```
Loading configuration...  
Initializing PySpin system...  
Found 2 cameras  
Camera 0: Detection (Serial: XXXXX)  
Camera 1: Inspection (Serial: XXXXX)  
Starting acquisition...
```

If calibration needed:

```
Starting auto-calibration...  
Detecting circles...  
Found 4 circles  
Calculating homography...  
✓ Calibration completed  
Saved homography_auto.pkl  
Saved homography_det_to_robot.pkl
```

Detection window will open showing live camera feed

Step 4: Verify Robot Completion (Terminal 1)

Watch Terminal 1 for:

```
✓ Calibration files updated  
Moving to home position...  
✓ Ready for operation
```

Robot will move to home: [1.5, 6.3, 45.5, 0, 39.2, 3.2]

Step 5: System Ready

Both terminals running: - Terminal 1: Robot controller (monitoring commands) - Terminal 2: Vision system (detection window visible)

Indicators of successful startup: - Detection window shows live camera feed - YOLO bounding boxes appear on detected objects - No error messages in either terminal - Robot at home position

Verification and Testing

Test 1: Camera Detection

In detection window: - Place an object in workspace - Verify YOLO bounding box appears - Check angle and dimensions display correctly

Test 2: Coordinate Transformation

Test tool:

```
python3 coordinate_tester.py
```

Follow prompts to test: 1. Click position in camera view 2. Verify transformed robot coordinates 3. Ensure coordinates within robot boundaries: - X: 88.9 to 382.0 mm - Y: 14.7 to 312.0 mm

Test 3: Robot Movement (LEFT CLICK)

WARNING: Start with object at safe distance from robot

1. **Left-click on object** in detection window
2. **Verify:**
3. Robot moves smoothly
4. Reaches correct position
5. No collision warnings
6. Returns to safe height

Test 4: Pick and Place (RIGHT CLICK)

1. **Place lightweight test object** in workspace
2. **Right-click on object**
3. **Verify sequence:**
4. Robot moves to position
5. Lowers to pick height
6. Gripper closes at correct angle
7. Picks up object

8. Returns to safe height
9. **Right-click target location**
10. **Verify:**
11. Robot moves to location
12. Lowers to place height
13. Gripper opens
14. Returns to safe height

Test 5: Inspection Mode (T KEY)

1. **Click object** to select
2. **Press T key**
3. **Verify:**
4. Inspection window opens
5. Robot tests multiple angles (0°, 90°, 180°, 270°)
6. Inspection camera shows close-up view
7. Crosshair centered on object

Test 6: Emergency Stop

CRITICAL SAFETY TEST:

1. **Initiate robot movement**
 2. **Press Ctrl+C in Terminal 1**
 3. **Verify:**
 4. Robot stops immediately
 5. Both programs exit cleanly
 6. No error states on robot
 7. **Recovery:**
 8. Restart both programs
 9. Verify normal operation
-

Troubleshooting

Robot Won't Connect

Error: `Failed to connect to robot at 192.168.1.151`

Solutions: 1. **Check network connection:** `bash ping 192.168.1.151` If no reply: - Verify Ethernet cable connected - Check robot power - Verify IP address in config.json - Check computer and robot on same subnet

1. **Check robot state:**
2. Use xArm Studio to connect
3. Clear any errors
4. Enable servos
5. Disconnect xArm Studio before running Python script
6. **Verify IP address:**
7. Robot display shows correct IP
8. Update config.json if changed

Cameras Not Detected

Error: `Found 0 cameras`

Solutions: 1. **Check USB connections:** `bash lsusb` Look for FLIR or Point Grey devices

1. **Check PySpin installation:** `bash python3 -c "import PySpin; print('PySpin OK')"`
2. **Restart USB:** `bash # Unplug and replug USB cables # Or restart udev: sudo udevadm control --reload-rules`
3. **Check permissions:** `bash # Add user to video group sudo usermod -a -G video $USER # Logout and login`

Calibration Fails

Error: `Could not detect 4 circles`

Solutions: 1. **Check marker visibility:** - Ensure good lighting - Increase contrast (dark background) - Verify markers are circular and flat - Check camera focus

1. **Adjust detection parameters** (if persistent): Edit `/home/user/arm-app/yolo-mouse-v2.py` line ~619: `python circles = cv2.HoughCircles( gray, cv2.HOUGH_GRADIENT, dp=1.2, minDist=100, param1=100, param2=30, # Try lowering to 20 minRadius=10, # Try adjusting maxRadius=50 # Try adjusting )`

2. Manual verification:

3. Run vision system alone
4. Check if circles are highlighted
5. Adjust marker size/position

YOLO Model Missing

Error: `Could not load model from ./models/best.pt`

Solution:

```
# Create models directory
mkdir -p /home/user/arm-app/models

# Add your trained YOLO model
# Copy best.pt to models/
cp /path/to/your/best.pt /home/user/arm-app/models/
```

Coordinates Out of Bounds

Error: `Target coordinates outside robot workspace`

Cause: Calibration mismatch or workspace definition incorrect

Solution: 1. **Verify workspace dimensions** in config.json 2. **Recalibrate system:** - Delete calibration files - Restart with correct marker placement 3. **Test with coordinate_tester.py**

Robot Moves to Wrong Position

Cause: Calibration error or coordinate transformation issue

Solution: 1. **Recalibrate immediately:** `bash rm homography_auto.pkl homography_det_to_robot.pkl` Restart system

1. **Verify marker placement:**
2. Must be exactly at workspace corners
3. Check measurements
4. **Test transformation:** `bash python3 coordinate_tester.py`

Inspection Camera Shows Nothing

Solutions: 1. **Check camera index** in config.json: `json "inspection_index": 1`

1. **Verify camera mounting:**

2. Camera facing downward
3. Proper offset configured
4. **Check inspection height** in config.json: `json "inspection_height": 103.4 // Adjust if needed`

Shared Memory Errors

Error: `Could not create shared memory`

Solution:

```
# Clean up shared memory
rm /dev/shm/DetectionData
rm /dev/shm/ClickData
rm /dev/shm/InspectData

# Restart system
```

Additional Resources

Documentation Files

- **Main README:** `/home/user/arm-app/README.md`
- **Error Guide:** `/home/user/arm-app/Guide/ERRORS_AND_FIXES.md`
- **Diagnostic Tool:** `/home/user/arm-app/Guide/diagnostic_check.py`

Running Diagnostics

```
cd /home/user/arm-app/Guide
python3 diagnostic_check.py
```

Contact Information

For additional support: - Check GitHub issues - Review ERRORS_AND_FIXES.md - Run diagnostic_check.py

Revision History

Version	Date	Changes	Author
1.0	2025-11-24	Initial SOP creation	Claude

Appendix: Quick Reference

Startup Commands

Option A: GUI Application (Calibration/Monitoring)

```
cd /home/user/arm-app && python3 arm-app-v1.1.py
```

Option B: Command-Line (Full Robot Control)

```
# Terminal 1
cd /home/user/arm-app && python3 xarm-motion-v1.7.py

# Terminal 2 (wait for robot calibration position)
cd /home/user/arm-app && python3 yolo-mouse-v2.py
```

Emergency Stop

```
Ctrl+C in either terminal
```

Recalibration

```
rm homography_auto.pkl homography_det_to_robot.pkl
# Restart system
```

File Locations

Configuration:	/home/user/arm-app/config.json
Calibration:	/home/user/arm-app/homography*.pkl
GUI Application:	/home/user/arm-app/arm-app-v1.1.py
Robot Controller:	/home/user/arm-app/xarm-motion-v1.7.py
Vision System:	/home/user/arm-app/yolo-mouse-v2.py
Documentation:	/home/user/arm-app/README.md

END OF SOP